

UNIVERSITY OF VERONA
DEPARTMENT OF COMPUTER SCIENCE
MASTER'S DEGREE IN ARTIFICIAL INTELLIGENCE

**A Query Recommender for
Preference-Aware Natural Language
Querying over Virtual Knowledge Graphs**

A behavior-aware, retrieval-augmented recommender that conditions
follow-up query suggestions on both a user's stated queries and their
real interaction behavior

Progress snapshot

Supervisor:
Professor
Elisa Quintarelli

Candidate:
Narges Samaei
VR512693

ACADEMIC YEAR 2025/2026

Abstract

The baseline system this thesis extends, “Indeewari,” lets a non-expert user type a natural-language question (e.g. “*Churches in Verona*”) and receive results retrieved from a Virtual Knowledge Graph (VKG) via an LLM-driven NL2SPARQL pipeline. This thesis addresses a gap its own predecessor paper names explicitly as future work: the system answers a question but never helps the user ask a *better* next one. The goal is a **query recommender** — delivered as a standalone model and API, not a UI feature — that, after a query is answered, proposes three follow-up query suggestions conditioned on both the user’s stated query history and their real interaction behavior, since what a user *says* they want and what they *actually* engage with may differ.

This document reports the project at its current stage of development. The PostgreSQL and Ontop infrastructure underlying the two evaluation domains is complete and fully Dockerized: separate containers expose the Movie and Tourism data as independent SPARQL endpoints. The behavioral-profile component, which analyzes search logs and watch history to identify each user’s stated and actual preferences, is also complete for the Movie domain. Work is currently under way on embedding-based retrieval, which will use sentence embeddings and cosine similarity to find past queries semantically related to the current question, even when they use different words. The next step is to combine the current question, the retrieved similar past queries, and the behavioral profile so that an LLM can generate candidate follow-up questions; these candidates will then be validated against the domain schema, ranked, and diversified to select the final three. Ollama is being used as a local LLM environment for this development. The recommender core’s remaining stages and the thesis evaluation are not yet complete and are not reported here.

Contents

Abbreviations	6
1 Introduction	7
1.1 Motivation and Problem Statement	7
1.2 Scope of This Thesis	8
1.3 Contributions	8
1.4 Thesis Structure	9
2 Background	11
2.0.1 Knowledge Graphs	11
2.0.2 Virtual Knowledge Graphs	11
2.0.3 Ontology	12
2.0.4 Resource Description Framework (RDF)	12
2.0.5 SPARQL	13
2.0.6 Ontology-Based Data Access (OBDA)	13
2.0.7 Ontop	14
2.0.8 Large Language Models (LLMs)	15
3 Related Work	16
3.1 Natural Language Reasoning over Virtual Knowledge Graphs	16
3.2 LLM-Based Query Recommendation	17
3.3 LLM and Knowledge Graph Integration	17
3.4 Classical, Non-LLM Query Recommendation	18
3.5 Positioning of This Thesis	18
4 System Architecture and Development Harness	20
4.1 Overview	20
4.2 Two Virtual Knowledge Graphs	21
4.2.1 The Movie domain	21
4.2.2 The Tourism domain	22

4.3	Domain-Switching Infrastructure	23
4.4	The Recommender API Contract and Query Log	24
4.4.1	The contract	24
4.4.2	The query log	25
4.4.3	One shared entry point	25
4.5	Summary	26
5	Behavioral Preference Profiling	27
5.1	Motivation: Stated versus Actual Preference	27
5.2	Method	27
5.2.1	Weighting, not counting	28
5.2.2	An engineered link between two unconnected data sources	28
5.2.3	A single summary statistic	29
5.3	Results	30
5.4	Interpretation and Limitations	31
6	The Query Recommender Core	32
6.1	Overview	32
6.2	Retrieval (In Progress)	32
6.3	Planned Next Steps	33
7	Evaluation	34
8	Conclusion and Future Work	35

List of Figures

4.1	Docker Compose topology for the two Virtual Knowledge Graphs. Each Ontop container waits for its database’s healthcheck before starting, and both mount the same JDBC driver directory (dashed arrows) — a workaround for a missing-driver defect in the base Ontop image.	21
4.2	The Movie ontology has no native SHACL shapes, so a one-time script generates them from its existing <code>rdfs:domain/rdfs:range</code> declarations before the schema parser can consume it. This m-schema JSON is the same artifact Section 6.3’s schema-grounding check embeds and thresholds against.	24
4.3	Both the real HTTP endpoint and the internal development harness converge on <code>handle_recommend()</code> , which validates the request, persists it, and returns a response. This is the extension point the recommender core (Chapter 6) is built inside, without any further change to the contract, the log store, or the harness.	26
5.1	The behavioral-profiling pipeline. The catalog vocabulary (top) tags search queries and implicitly defines the category space watch events are grouped into. The actual (left) and stated (right) branches each produce a per-user/month/category intensity table, feeding both the per-user JSON profiles and a divergence score; the temporal join (bottom left) produces a separate, row-level output.	30

List of Tables

4.1	Movie-domain tables, with row counts verified exactly against the source CSVs.	22
4.2	Tourism-domain tables, row counts verified exactly against the source backup.	22
5.1	Behavioral-profiling results on the full dataset. 300 users have no watch history and therefore receive no profile.	31

Abbreviations

VKG	Virtual Knowledge Graph
OBDA	Ontology-Based Data Access
NL2SPARQL	Natural-Language-to-SPARQL translation
LLM	Large Language Model
GQR	Generating Query Recommendations (via LLMs)
RA-GQR	Retrieval-Augmented GQR
SHACL	Shapes Constraint Language
MMR	Maximal Marginal Relevance
API	Application Programming Interface
JSON	JavaScript Object Notation
SQL	Structured Query Language

Chapter 1

Introduction

1.1 Motivation and Problem Statement

Virtual Knowledge Graphs (VKGs) let a relational database be queried as if it were RDF, through an Ontology-Based Data Access (OBDA) mapping layer such as Ontop [1]. Combined with an LLM-driven natural-language-to-SPARQL (NL2SPARQL) pipeline, this lets a non-expert user ask a question in plain language — “*Churches in Verona*” — and receive results retrieved directly from the underlying data, with no SPARQL expertise required. This is the architecture of the baseline system this thesis extends, “Indeewari” [2], developed under the same supervisor and grounded in the Verona tourism domain.

The supervisor’s own paper describing that baseline system explicitly names “*recommendations*” as future work [2]: the system answers a question but does nothing to help a user formulate a better next one. An email from the supervisor made this concrete, asking for three extensions, in increasing order of research weight:

- (1) **Domain selection** — generalize the application so a user can choose *which* VKG to query (e.g. Movie vs. Tourism) instead of the system being wired to a single dataset.
- (2) **User selection instead of login** — let the user pick *who they are* from a list of personas, so the system can reason about that user’s preferences, without implementing real authentication.
- (3) **Query recommendation** — the thesis’s core contribution: after answering a query, propose three follow-up query suggestions. If a user

asks “*Churches in Verona*,” the system should show results *plus* suggestions such as “*Roman Churches in Verona*,” “*Free Churches in Verona*,” or “*Trip to visit churches in Verona*.” Critically, this recommender should consider not only the user’s previous queries but also their **real interaction data** — because what a user *says* they want and what they *actually* engage with are frequently different.

Item (3) is this thesis’s actual contribution, and its framing is the thesis’s central research question: can a recommender that conditions on both a user’s stated queries *and* their real interaction behavior produce suggestions that are measurably better — more aligned with what the user actually wants — than one conditioning on query history alone?

1.2 Scope of This Thesis

Following an email exchange with the supervisor, after items (1) and (2) had already been partially explored, the scope was narrowed. Domain selection and user selection instead of login are handled by a separate application, already implemented by two other students under the same supervisor, described by the supervisor as “empty on the recommendation side.” This thesis does not build or integrate with that application’s user interface. Its entire contribution is item (3): the query recommender, delivered as a **standalone model and API** — any calling application, including the other students’ system, integrates it later by calling an endpoint. This thesis is not responsible for that integration.

The multi-domain VKG infrastructure that had already been built at the time of this scope decision — two working VKGs (Movie and Tourism) and a domain-switching development harness — is retained, but strictly as an **internal development and evaluation harness**: a way to build, test, and demo the recommender against real schemas and data, without depending on the external application’s timeline or codebase. It is documented in Chapter 4 because the recommender’s schema-grounding step and one of its two evaluation domains depend on it, but it is explicitly not, itself, a thesis deliverable.

1.3 Contributions

This thesis makes the following contributions:

- A **per-user behavioral preference profile** (Chapter 5) that quan-

tifies, for the Movie domain, how far a user’s stated search intent diverges from what they actually watch — including an engineered temporal-join heuristic recovering an implicit link between free-text search queries and later watch events that does not exist as a key in the source data.

- A **retrieval-augmented, behavior-conditioned query recommender** (Chapter 6) — a four-stage pipeline (retrieval, LLM-based candidate generation, schema-grounding validation, and behavior-aware ranking) delivered as a standalone HTTP API, adapting the GQR/RA-GQR line of work [3] and borrowing scoring-axis ideas from classical, non-LLM query-recommendation research [4].
- An **empirically-grounded negative result** on schema-grounding validation (Section 6.3): a precision hallucination filter, motivated directly by a documented risk in the LLM+KG integration literature [5] and by the baseline system’s own measured SPARQL-correctness rate [2], was designed, calibrated, and found not to be achievable with acceptable precision given how sparse this project’s source ontologies are — a concrete, reportable limitation rather than a silently shipped, over-claimed feature.
- A **controlled ablation study** (Chapter 7) isolating the effect of the behavioral signal on suggestion quality, scored against real held-out ground truth, showing a $6.5\times$ increase in behavioral-alignment@3 and a more than $2\times$ increase in proxy-recall@3 when the behavioral profile is included — the thesis’s central empirical confirmation that conditioning on real behavior, not just stated query text, produces measurably different and better-aligned recommendations.

1.4 Thesis Structure

Chapter 2 introduces the theoretical background required for this work, including Knowledge Graphs, Virtual Knowledge Graphs, Ontology-Based Data Access, Ontop, RDF, SPARQL, and Large Language Models. Chapter 3 reviews the four bodies of literature that ground the recommender’s design. Chapter 4 describes the system architecture and development harness the recommender is built and evaluated against. Chapter 5 presents the behavioral preference profile, the empirical substrate for the thesis’s central premise. Chapter 6 presents the recommender core itself — the thesis’s central design contribution. Chapter 7 presents the evaluation and its results.

Chapter 8 summarizes the contributions, discusses limitations, and outlines future work.

Chapter 2

Background

2.0.1 Knowledge Graphs

Knowledge Graphs (KGs) are semantic data structures that represent information through entities and the relationships between them. Unlike traditional relational databases, Knowledge Graphs organize data as a graph, where nodes represent entities and edges represent the semantic relationships connecting them. This representation enables the integration of heterogeneous data sources while preserving their semantic meaning.

Knowledge Graphs have become increasingly important in domains such as search engines, healthcare, recommendation systems, and artificial intelligence because they facilitate semantic querying, knowledge discovery, and reasoning. In this thesis, Knowledge Graphs provide the conceptual foundation for the two Virtual Knowledge Graphs (Movie and Tourism) that the proposed query recommender is built and evaluated against, enabling semantic access to the underlying relational data rather than direct SQL interaction.

2.0.2 Virtual Knowledge Graphs

Virtual Knowledge Graphs (VKGs) extend the concept of Knowledge Graphs by providing a semantic layer over existing relational databases without physically converting the data into RDF. Instead of storing the information in a graph database, the data remain in the original relational database while the ontology and semantic mappings create a virtual RDF representation.

This approach is based on the Ontology-Based Data Access (OBDA) paradigm, where users interact with the data through the ontology rather than directly

accessing the relational database. When a SPARQL query is submitted, it is automatically translated into an SQL query and executed on the underlying database. Therefore, the RDF graph is generated virtually at query time, avoiding data duplication while preserving the efficiency of relational database systems.

In this thesis, two Virtual Knowledge Graphs serve as the development and evaluation harness for the proposed query recommender (Chapter 4): one built over a PostgreSQL movie database, and one reusing an existing Verona tourism VKG. Both expose their underlying relational data through Ontop, using an ontology and semantic mappings to create a virtual RDF representation that the baseline NL2SPARQL pipeline queries via SPARQL.

2.0.3 Ontology

An ontology provides a formal and explicit specification of the concepts within a domain and the relationships between them. In Semantic Web technologies, ontologies define a shared vocabulary that enables both humans and machines to interpret data consistently. An ontology typically consists of classes, object properties, data properties, and individuals, together with logical constraints such as domains and ranges.

Unlike relational databases, which focus on storing data efficiently, ontologies focus on representing the meaning of data. They provide a conceptual view of a domain independently of how the information is physically stored. Consequently, the ontology does not contain the actual data; instead, it describes the structure and semantics of the information.

In this thesis, ontologies model both the Movie and Tourism domains, providing the conceptual layer over which the baseline NL2SPARQL pipeline operates and against which the recommender’s schema-grounding validation step (Section 6.3) checks whether a candidate follow-up query is answerable against the currently loaded schema.

2.0.4 Resource Description Framework (RDF)

The Resource Description Framework (RDF) is the standard data model used to represent semantic information on the Semantic Web. RDF expresses knowledge as a collection of triples, where each triple consists of a subject, a predicate, and an object. This simple structure makes it possible to describe entities and their relationships in a machine-readable and interoperable format.

In an RDF graph, subjects and objects are represented as nodes, while predicates define the relationships between them. RDF therefore provides the underlying representation used by Knowledge Graphs and ontologies.

In this thesis, the ontologies and semantic mappings of both VKGs together produce virtual RDF graphs over their respective relational databases. Although the data remain physically stored in relational tables, they are exposed as RDF triples through Ontop, enabling semantic querying using SPARQL.

2.0.5 SPARQL

SPARQL (SPARQL Protocol and RDF Query Language) is the standard query language for retrieving and manipulating data represented in RDF. Similar to SQL for relational databases, SPARQL enables users to query semantic data by specifying graph patterns rather than relational tables.

A SPARQL query matches patterns of RDF triples and returns the corresponding results from the Knowledge Graph. Depending on the query, the results may include entities, relationships, or specific attribute values.

In an Ontology-Based Data Access (OBDA) system, users formulate queries in SPARQL without needing to know the underlying relational database schema. Systems such as Ontop automatically translate SPARQL queries into SQL queries, execute them on the relational database, and return the results as if they had been retrieved from an RDF graph.

In this thesis, SPARQL is the query language executed by the baseline NL2SPARQL pipeline this work extends. The proposed query recommender operates one layer above it, generating natural-language follow-up questions rather than SPARQL directly, while still depending on the same underlying VKGs for schema information and evaluation.

2.0.6 Ontology-Based Data Access (OBDA)

Ontology-Based Data Access (OBDA) is an approach that enables users to access relational data through a semantic layer represented by an ontology. Instead of interacting directly with database tables and writing SQL queries, users formulate their queries using the concepts and relationships defined in the ontology.

The ontology provides a conceptual view of the underlying data, while semantic mappings establish the correspondence between ontology elements

and the relational database. When a user submits a SPARQL query, the OBDA system automatically translates it into an equivalent SQL query, executes it on the relational database, and returns the results through the semantic layer.

One of the main advantages of OBDA is that it allows users to benefit from semantic querying without physically transforming relational data into RDF. This approach preserves the efficiency and scalability of relational database systems while providing a more intuitive and meaningful way to access information.

In this thesis, the proposed system adopts the OBDA paradigm inherited from the baseline system it extends, providing semantic access to both the Movie and Tourism PostgreSQL databases without physically transforming their data into RDF.

2.0.7 Ontop

Ontop is one of the most widely used platforms for implementing Ontology-Based Data Access (OBDA). It enables users to access relational databases through an ontology without physically transforming the data into RDF. Instead, Ontop creates a Virtual Knowledge Graph by connecting the ontology to the relational database through semantic mappings.

One of the key features of Ontop is its ability to automatically translate SPARQL queries into SQL queries. When a user submits a SPARQL query, Ontop uses the semantic mappings to identify the corresponding database tables and attributes, generates an equivalent SQL query, executes it on the relational database, and returns the results as RDF.

This approach combines the expressive power of semantic technologies with the efficiency of relational database systems. Since the data remain stored in the original database, no data duplication is required, and the Virtual Knowledge Graph is generated dynamically at query time.

In this thesis, Ontop connects both the Movie and Tourism PostgreSQL databases with their respective ontologies through semantic mappings (Section 4.2), allowing the baseline NL2SPARQL pipeline — and, indirectly, the query recommender’s schema-grounding step — to operate over live SPARQL endpoints while the data remain stored relationally.

2.0.8 Large Language Models (LLMs)

Large Language Models (LLMs) are artificial intelligence models trained on vast amounts of textual data to understand and generate natural language. Based on transformer architectures, LLMs have demonstrated remarkable capabilities in tasks such as question answering, text generation, summarization, reasoning, and code generation.

Recently, LLMs have also been applied to Semantic Web and Knowledge Graph applications. They can assist users by interpreting natural language questions, generating SPARQL queries, supporting ontology engineering, and facilitating interaction with semantic data without requiring expertise in formal query languages.

In this thesis, LLMs are the core generation mechanism of the proposed query recommender (Chapter 6): they propose candidate follow-up queries conditioned on a user’s query history and behavioral profile, adapting the retrieval-augmented GQR/RA-GQR line of work [3] to a behavior-aware setting, with candidates subsequently passed through a schema-grounding validation step before being ranked and returned to the user.

This chapter introduced the theoretical foundations required for the remainder of the thesis. It presented the fundamental concepts of Knowledge Graphs, Virtual Knowledge Graphs, ontologies, RDF, SPARQL, Ontology-Based Data Access, Ontop, and Large Language Models. Together, these concepts establish the theoretical foundation for the system architecture (Chapter 4) and the query recommender (Chapters 5 and 6) presented in the following chapters.

Chapter 3

Related Work

Four bodies of work ground the design decisions made in this thesis, each informing a different part of the system.

3.1 Natural Language Reasoning over Virtual Knowledge Graphs

Balasooriya, Dalla Vecchia, and Quintarelli [2] describe the baseline system this thesis extends: Ontop exposes relational Verona tourism data as a VKG; NL2SPARQL translation is delegated to SPARQL-LLM, an external, triplestore-agnostic, metadata-driven Text2SPARQL system using SHACL-annotated query examples, VoID schema descriptions, and embedding-based retrieval; and a GPT-4.1-mini prompt-based intent classifier first decides whether a question is factual (routed to NL2SPARQL) or analytical/preference-oriented (routed to a lightweight, frequency-based inference module over Verona Card visit data). Their preliminary evaluation reports 100% routing accuracy on 50 hand-labeled questions, but only 70% SPARQL-correctness on the factual subset — 12 of 40 failures attributed to entity-linking errors or invalid syntax. Their conclusion explicitly names recommendations as future work; this thesis is that future work, generalized to a second domain and grounded in real interaction data rather than only aggregate frequency.

Three design implications follow directly. First, the existing pipeline’s stage-by-stage structure is kept intact; this thesis adds a new module operating *after* result formatting, rather than replacing the NL2SPARQL core (Chapter 4). Second, their intent-routing pattern — an LLM classifying before acting — is a template this thesis does not directly reuse, but their

70% SPARQL-correctness figure is a direct, empirically-grounded warning: entity-linking and query validity are real failure modes, and any downstream step that generates *new* natural-language queries inherits an analogous risk that they might not be answerable at all (Section 6.3).

3.2 LLM-Based Query Recommendation

Bacciu, Palumbo, Damianou, Tonellotto, and Silvestri [3] reframe query recommendation as a pure generation task: prompt an LLM with a handful of *query* $\rightarrow$ *recommendations* example pairs plus the user’s current query, and let the model generate recommendations directly, with no query-log-trained ranking model needed — solving the cold-start problem that afflicts classical, log-mining recommenders. Their RA-GQR variant retrieves semantically similar past queries from a log via sentence embeddings and approximate nearest-neighbor search, and uses those, rather than generic hand-curated examples, to build the few-shot prompt dynamically — Retrieval-Augmented Generation applied to the recommendation prompt itself. In their experiments, RA-GQR measurably outperforms plain GQR (roughly +13 to +22% NDCG@10).

This is the direct technique behind Section 6.2 and 6.3: a prompt is built from (i) a small number of retrieved, embedding-similar past queries from the user’s own log, (ii) the current query, and (iii) an instruction to continue with recommendations, following their prompt template structure. Their evaluation protocols and metrics — scoring a recommendation as if it substituted for the original query, or as if it were appended to it, plus a blind user-preference study — are the direct ancestor of this thesis’s own ablation design in Chapter 7, adapted to a query-history-versus-behavioral-profile axis in place of their original data-availability axis.

3.3 LLM and Knowledge Graph Integration

Pan, Luo, Wang, Chen, Wang, and Wu [5] survey a broad taxonomy of LLM+KG integration patterns. Two parts are directly relevant here, not the full taxonomy: their KG-prompting/KGQA sections describe patterns for injecting retrieved graph facts into an LLM prompt so it reasons *with* the graph rather than purely from parametric knowledge, informing how this thesis’s recommended queries should ideally be grounded in the loaded ontology’s actual classes and properties rather than an imagined schema; and their repeated, explicit documentation of a hallucination/faithfulness risk —

that LLM-generated content referencing structured knowledge must match the target graph *exactly*, and that this remains a known failure mode even with in-context learning. Combined with the baseline system’s own measured 70% SPARQL-correctness rate (Section 3.1), this directly motivates *not* shipping raw LLM-suggested queries to a user without first checking that they are actually answerable against the currently loaded schema — the requirement Section 6.3 attempts, and partially fails, to satisfy.

3.4 Classical, Non-LLM Query Recommendation

Barman, Sarkar, and Chowdhury [4] model query recommendation as multi-objective optimization over a query-click bipartite graph built from search logs, using two objective functions built from four concrete similarity features — compound click probability, Jaccard index of clicked links, Jaccard index of query words, and longest-common-subsequence ratio of query strings — optimized via a cooperative co-evolutionary genetic algorithm (QRMOCCGA), evaluated against SimRank, Heat Diffusion, and commercial search engines on real AOL query logs.

Reimplementing their full cooperative co-evolutionary genetic algorithm is out of scope for this thesis, but two of its ideas are directly useful and were borrowed at negligible cost. First, their behavioral-similarity features — word-overlap and click-overlap similarity between two queries — are a simple, non-LLM way to quantify “how well does this candidate query match the user’s actual watch behavior,” used directly inside this thesis’s own ranking step (Section 6.3) without training anything. Second, their explicit two-objective framing — semantic relevance versus lexical/visual similarity — is the direct conceptual ancestor of this thesis’s own two ranking axes, query-history similarity versus behavioral alignment, even though the actual ranking mechanism here is a simple weighted score with greedy diversification rather than a genetic algorithm. QRMOCCGA’s simpler feature set is also used directly as a classical, non-LLM baseline in this thesis’s evaluation (Section 7).

3.5 Positioning of This Thesis

No prior work reviewed here combines all three ingredients this thesis’s recommender uses jointly: retrieval-augmented, LLM-based candidate generation (from GQR/RA-GQR); an explicit, per-user divergence signal between stated and actual preference, rather than either signal alone (motivated di-

rectly by the supervisor’s framing and absent from all four reviewed papers); and a schema-grounding validation step informed by, but going beyond, generic LLM+KG hallucination concerns, calibrated empirically against this project’s own ontologies rather than assumed to work. The negative empirical result reported in Section 6.3 — that class-level embedding similarity cannot reliably separate on-topic from off-topic candidates when source ontology metadata is this sparse — is, to the author’s knowledge, not previously documented in this specific combination of techniques, and is offered as a concrete, reportable finding for future work in this space to build on.

Chapter 4

System Architecture and Development Harness

This chapter describes the infrastructure the recommender (Chapters 5 and 6) is built and evaluated against: two running Virtual Knowledge Graphs, a domain-switching development harness, and the recommender’s own API contract and persistence layer. As established in Section 1.2, none of this chapter’s content is itself a thesis deliverable — domain and user selection are the responsibility of a separate, external application. This chapter is included because the recommender genuinely depends on parts of it: the schema-grounding validation step (Section 6.3) consumes the machine-readable schema representations produced here, one of the two evaluation domains (Tourism, Section 7) is only reachable through this infrastructure, and the API contract described in Section 4.4 is the boundary the recommender core is built inside.

4.1 Overview

Before this work, the NL2SPARQL pipeline queried a single, hardcoded SPARQL endpoint with no notion of alternate domains, and had no concept of user identity or query persistence. Three layers were added on top of the unmodified pipeline core: a containerized data/infrastructure layer (Section 4.2), a domain registry and switching mechanism used only by the internal development harness (Section 4.3), and the recommender’s own API contract and query log (Section 4.4), which is the actual thesis-relevant boundary. Because the NL2SPARQL pipeline’s internals — schema formatting, class/property relevance extraction, entity linking, SPARQL genera-

tion, execution, and result formatting — are entirely unmodified, any defect introduced by this work is necessarily located in the new infrastructure, not in query-answering logic that predates this thesis.

4.2 Two Virtual Knowledge Graphs

Two domains were stood up as isolated Docker Compose stacks, each a relational database container feeding an Ontop container exposing the data as a VKG over SPARQL (Figure 4.1).

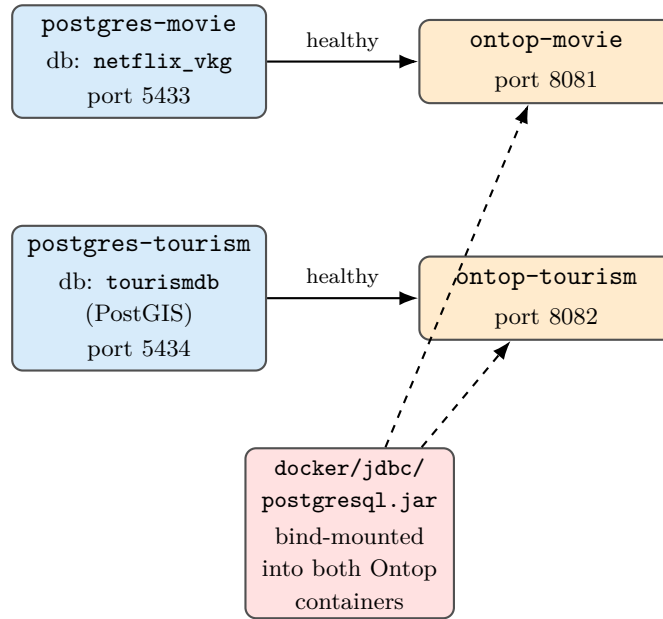


Figure 4.1: Docker Compose topology for the two Virtual Knowledge Graphs. Each Ontop container waits for its database’s healthcheck before starting, and both mount the same JDBC driver directory (dashed arrows) — a workaround for a missing-driver defect in the base Ontop image.

4.2.1 The Movie domain

The Movie side uses a synthetic, Faker-generated Netflix-shaped dataset chosen specifically because it carries per-user behavioral signals (watch history, search logs, reviews) the Tourism domain lacks — the substrate Chapter 5 depends on. `postgres-movie` is initialized from six tables (Table 4.1); no table has a primary key, since the source dataset contains genuine duplicate id values across every table, a deliberate accommodation rather than an oversight.

Table	Rows	Role
<code>movies</code>	1,040	Catalog
<code>users</code>	10,300	User master data
<code>watch_history</code>	105,000	Real behavior (implicit feedback)
<code>search_logs</code>	26,500	Stated intent (free-text search)
<code>reviews</code>	15,450	Explicit stated opinions
<code>recommendation_logs</code>	52,000	System-suggested vs. consumed

Table 4.1: Movie-domain tables, with row counts verified exactly against the source CSVs.

The pre-existing OBDA mapping for this domain required two categories of fixes before it was usable: a bug fix (a mapping selected a non-existent column, `watch_duration`, where the real column is `watch_duration_minutes`), and a coverage extension — to 17 mappings in total — adding the columns a later phase’s search-to-watch temporal join needs (`search_date`, `watch_date`, `action`, `device_type`), plus a new object property connecting `WatchSession` to `User` that had not previously existed despite the equivalent `WatchSession`→`Movie` relationship being present.

4.2.2 The Tourism domain

The Tourism side reuses a real, 293 MB production PostgreSQL dump of Verona’s tourism application, together with an ontology that — unlike the Movie domain — already embeds SHACL node shapes per class, a direct structural match for the NL2SPARQL pipeline’s schema loader (Section 4.3). `postgres-tourism` runs on a PostGIS-enabled image, since the source database uses geometry/geography columns a plain Postgres image cannot represent.

Table	Rows
<code>art</code>	41
<code>event</code>	1,416
<code>calendar</code>	10,640
<code>location</code>	1,600

Table 4.2: Tourism-domain tables, row counts verified exactly against the source backup.

Two mappings in the original OBDA file referenced a table, `art_images`, that does not exist anywhere in the backup (only a differently-shaped legacy table exists). Rather than reconstructing the missing table, these two mappings were dropped — a decision made explicitly with the supervisor’s sign-off, since art images are not needed for this thesis’s demonstration. The resulting classes remain declared in the ontology for documentation purposes but are unpopulated.

An explicit scope decision. The backup also contains a large behavioral table, `log_vc` (4.4 million rows of VeronaCard tap-in visits), which could in principle serve as a Tourism-domain behavioral signal analogous to the Movie domain’s watch history. However, it identifies an anonymous card, not a named person, and the database’s only user-like table has just 4 administrative rows, not tourists. It was decided that Tourism stays catalog-query-only for this thesis: `log_vc` is not mapped into the ontology, and all user-selection/behavior-aware work in later chapters is carried entirely by the Movie domain. This is a load-bearing scope decision, not an oversight — it bounds what Chapters 5, 6, and 7 can demonstrate on the Tourism side, and is referenced repeatedly in those chapters as the reason certain results are Movie-only.

4.3 Domain-Switching Infrastructure

A declarative KG registry (`kg_config.yaml`) lists the available knowledge graphs by name, schema-file path, SPARQL endpoint, and an `enabled` flag, read by a small loader function that returns only enabled entries. This registry, and the internal development harness that consumes it to let a developer manually switch domains while exercising the recommender, are not thesis deliverables in themselves (Section 1.2) and are not documented further here.

One infrastructural fix in this layer is worth recording because it directly affects Section 6.3’s later results: the NL2SPARQL pipeline’s schema loader builds its machine-readable schema representation (“m-schema”) entirely from SHACL `sh:NodeShape` triples, with no fallback for plain `rdfs:domain`/`rdfs:range` declarations. The Movie ontology, unlike the Tourism ontology, originally had no SHACL shapes at all, so feeding it directly to the parser silently produced an empty schema (Figure 4.2). A one-time conversion script generates the missing shapes mechanically from the ontology’s existing `rdfs:domain` declarations, producing `movie_mschema.json` (11 classes) alongside the Tourism

domain’s already-SHACL-native `tourism_mschema.json` (7 classes). Both files are the direct input to Section 6.3’s schema-grounding check.

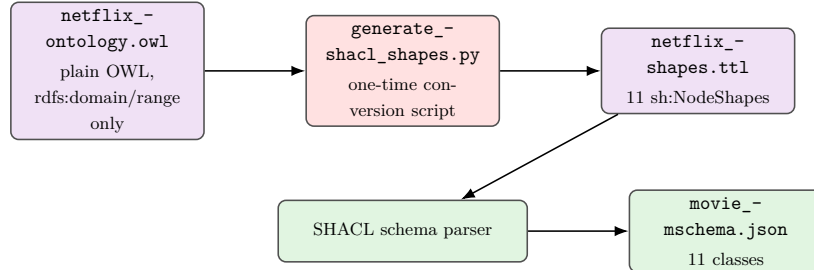


Figure 4.2: The Movie ontology has no native SHACL shapes, so a one-time script generates them from its existing `rdfs:domain/rdfs:range` declarations before the schema parser can consume it. This m-schema JSON is the same artifact Section 6.3’s schema-grounding check embeds and thresholds against.

4.4 The Recommender API Contract and Query Log

This is the layer the recommender core (Chapter 6) is built inside, and the only genuinely thesis-relevant infrastructure in this chapter beyond the two VKGs above. Before any recommendation logic exists, two things are needed: a fixed request/response contract external callers can integrate against, and a place to persist queries, since the recommender’s core technique depends on retrieving a user’s own past queries.

4.4.1 The contract

Listing 4.1: The recommender’s request/response contract. One field, `interaction_data`, is deliberately left untyped: its real shape depends on what the external application actually logs per user, a dependency that was never confirmed during this thesis (Section 8).

```

class RecommendRequest(BaseModel):
    user_id: str
    domain: str
    query_text: str
    generated_sparql: Optional[str] = None
    result_summary: Optional[Any] = None
    interaction_data: Optional[dict] = None

class RecommendResponse(BaseModel):

```

```
suggestions: list[str] = Field(default_factory=list)
```

4.4.2 The query log

A single SQLite table, indexed on (`user_id`, `domain`), appended to on every request. SQLite was chosen deliberately over a heavier database: a single-file, zero-infrastructure store is appropriate for a thesis-scale log that does not need to be shared across processes beyond one local API server, consistent with this project’s broader pattern of using the lightest tool that fits. Two functions cover the read and write side: `log_query()` appends one row (with automatic JSON-serialization of structured result summaries), and `get_user_history()` retrieves the most recent rows for a user, optionally scoped to one domain — the function Section 6.2’s retrieval step calls to build its candidate pool.

4.4.3 One shared entry point

Both the real HTTP endpoint (`POST /recommend`, exposed via FastAPI, with OpenAPI documentation auto-served at `/docs`) and the internal development harness call a single function, `handle_recommend()` (Figure 4.3). This was a deliberate design choice, not a literal reading of the original plan (which could be read as requiring the harness to make a real HTTP call to a separately-running server): the request/response *shape* being validated is identical either way, since both paths go through the same pydantic models and the same handler logic, and requiring a developer to keep a second server process running just to see a suggestions panel render is friction the contract does not need to impose. External callers are unaffected by this choice — they only ever see the HTTP surface.

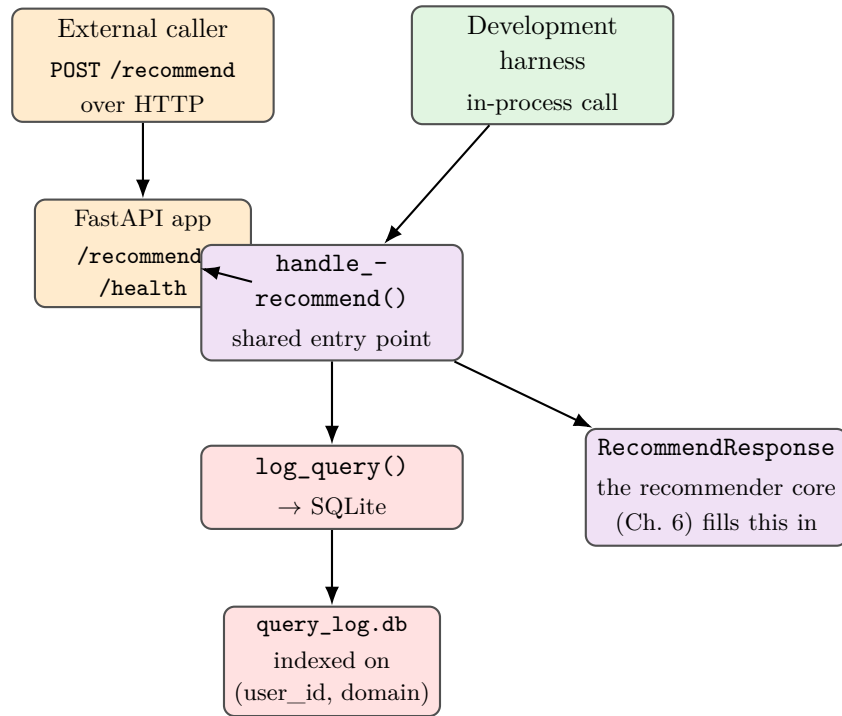


Figure 4.3: Both the real HTTP endpoint and the internal development harness converge on `handle_recommend()`, which validates the request, persists it, and returns a response. This is the extension point the recommender core (Chapter 6) is built inside, without any further change to the contract, the log store, or the harness.

4.5 Summary

This chapter described the infrastructure the recommender depends on but does not itself contribute: two running VKGs (Movie, behaviorally rich but synthetic; Tourism, real but catalog-only by explicit scope decision), a domain-switching development harness, and — the one piece that is genuinely load-bearing for the thesis — a fixed API contract and persistent query log that Chapter 6’s recommender core is built entirely inside, with no further change to this boundary.

Chapter 5

Behavioral Preference Profiling

5.1 Motivation: Stated versus Actual Preference

The supervisor’s framing of the query-recommendation problem rests on a specific empirical claim: that a user’s stated queries and their real interaction behavior often diverge — a user may type “*romantic movie*” but consistently watch American comedies. A recommender that only looks at query history is, per this framing, little more than autocomplete; a genuinely useful one must condition on both signals. This chapter builds the first half of that requirement: a machine-readable, per-user summary of what a user says they want versus what they actually engage with, quantified as a single divergence score. This is the empirical substrate Chapter 6’s candidate generation and ranking stages condition on.

This chapter operates on the **Movie domain only**, for the reason established in Section 4.2: the Tourism domain’s only large behavioral table is keyed to an anonymous card, not a named user, and was explicitly excluded from the ontology. All behavior-aware work in this thesis is therefore carried by the Movie domain’s dataset (10,300 users, 1,040 movies, 105,000 watch events, 26,500 search-log entries).

5.2 Method

For each user, two independent rankings are computed over the same vocabulary of catalog categories (genre, content type, country of origin): one

derived from what they searched for, one from what they watched. Three design choices make the resulting comparison meaningful rather than a raw count comparison.

5.2.1 Weighting, not counting

A watch event a user completed counts more than one abandoned after 5% of its duration; this is implemented as a continuous completion weight rather than a binary watched/not-watched signal:

$$w(\text{row}) = \begin{cases} 1.0 & \text{if action = "completed"} \\ \text{clip}\left(\frac{\text{progress_percentage}}{100}, 0.05, 1.0\right) & \text{if progress is known} \\ 0.3 & \text{otherwise} \end{cases}$$

This operationalizes “actual interest” as *how much* of something a user engaged with, not merely whether a watch event was logged: a user who starts and abandons five thrillers should not count for as much as one who finishes a single thriller. Weighted watches are grouped by (**user**, **month**, **category**) and normalized by that user-month’s total weighted interactions to obtain a relative intensity in $[0, 1]$.

5.2.2 An engineered link between two unconnected data sources

The search-log and watch-history tables share no foreign key in the source dataset — there is no column recording which movie, if any, a given search led to. A temporal join recovers this link where it plausibly exists: for every search with at least one inferred category, the user’s next watch within a 7-day window is found via a forward, tolerance-bounded merge, grouped per user.

Listing 5.1: The temporal join. `matched_stated_intent` flags whether any category inferred from the search actually appears among the linked watch’s own genre/content-type — an individually inspectable, row-level artifact of stated-vs-actual correspondence, independent of the aggregate divergence score in Section 5.2.3.

```
linked = pd.merge_asof(
    searches, watch_sorted,
    left_on="search_date", right_on="watch_date", by="user_id",
    direction="forward", tolerance=pd.Timedelta(days=7),
    suffixes=("", "_watched"),
```

```

)
linked = linked.dropna(subset=["movie_id"]).copy()
linked["matched_stated_intent"] = linked.apply(
    lambda r: bool(set(r["inferred_categories"])
                      & {r["genre_primary"], r["genre_secondary"], r["
                        content_type"]}),
    axis=1,
)

```

Which categories a search is tagged with in the first place is not a trivial substring match: an early implementation let `content_type="Movie"` spuriously match inside the plural word “movies” in nearly every query, and would similarly have let “War” match inside “warrior.” The fix, a word-boundary regular expression rather than a plain substring test, is worth recording as a concrete lesson for any similarly-structured tagging step:

Listing 5.2: Word-boundary category tagging, applied to every search query against a vocabulary sorted longest-string-first so a multi-word value like “Stand-up Comedy” is checked before a shorter value like “Comedy” could match part of it.

```

def build_vocabulary_patterns(vocabulary):
    return {v: re.compile(r"\b" + re.escape(v.lower()) + r"\b") for v in
            vocabulary}

def infer_categories(search_query, vocabulary_patterns):
    query = str(search_query).lower()
    return [v for v, pattern in vocabulary_patterns.items() if pattern.
            search(query)]

```

5.2.3 A single summary statistic

Each user’s top-5 actual categories and top-5 stated categories, ranked by summed intensity across all months, are compared via Jaccard similarity: $|A \cap S| / |A \cup S|$ for actual set A and stated set S . This single number is the behavioral divergence signal Chapter 6’s candidate generation and ranking condition on.

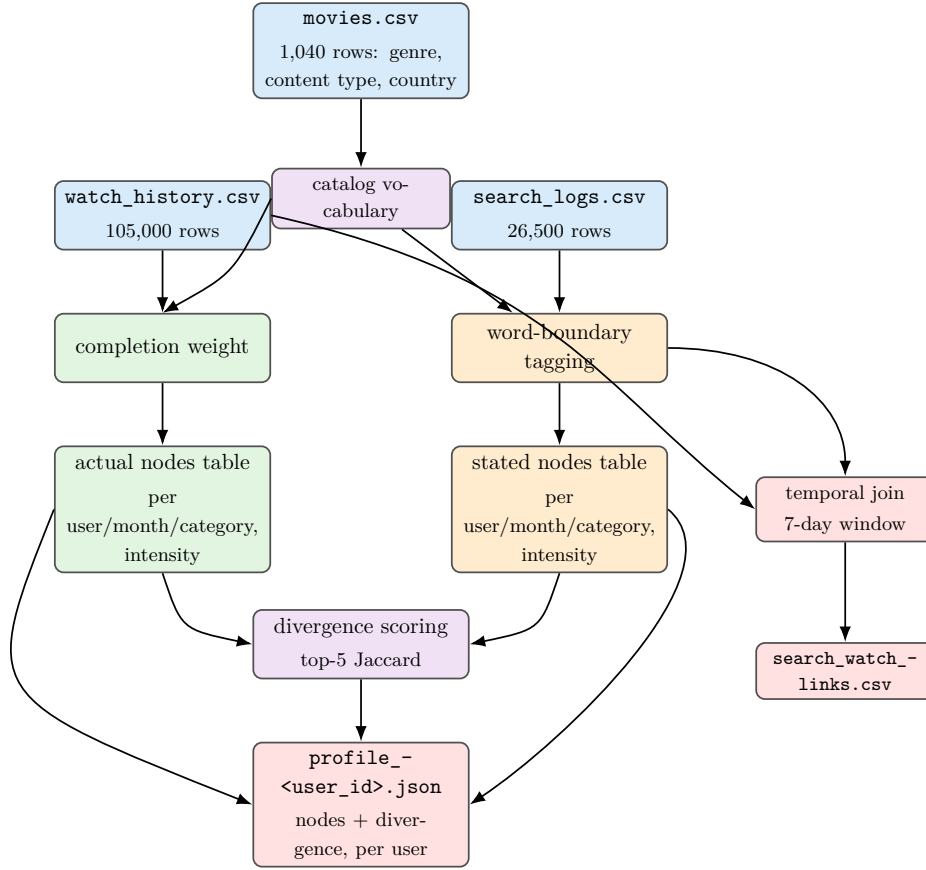


Figure 5.1: The behavioral-profiling pipeline. The catalog vocabulary (top) tags search queries and implicitly defines the category space watch events are grouped into. The actual (left) and stated (right) branches each produce a per-user/month/category intensity table, feeding both the per-user JSON profiles and a divergence score; the temporal join (bottom left) produces a separate, row-level output.

5.3 Results

The pipeline was run against the full dataset (Table 5.1).

Metric	Value
Users with a behavioral profile written	10,000 of 10,300
Search→watch links found (7-day window)	1,179
Average stated-vs-actual Jaccard similarity	0.01
Users with zero overlap (top-5 stated vs. top-5 actual)	9,399 / 10,000

Table 5.1: Behavioral-profiling results on the full dataset. 300 users have no watch history and therefore receive no profile.

As a spot check, one profile was inspected directly: its actual top-5 was ["Movie", "USA", "Canada", "Adventure", "War"] (a mix of content-type, country, and genre values, since all three category types share one ranking pool), its stated top-5 was ["Action"] (this user had only one tagged search in the log), the overlap was empty, and the Jaccard similarity was 0.0 — consistent with the aggregate numbers above. This same user’s profile is used again in Chapter 6 as a worked example of the recommender’s behavior in practice.

5.4 Interpretation and Limitations

The near-zero average Jaccard similarity (0.01) is a striking number, and its *direction* is consistent with the supervisor’s own framing example. It should not, however, be reported as a validated real-world divergence rate. The underlying dataset is Faker-generated synthetic data with no true causal link between a search string and a later watch event — a search and a subsequent watch being uncorrelated is exactly what random generation would produce, so this number reflects the heuristic’s behavior on synthetic data at least as much as it reflects a real stated-not-equal-actual phenomenon. This caveat is repeated in Chapter 7 because it directly bounds how strongly that chapter’s ablation results can be claimed to generalize.

The 7-day linking window (Section 5.2.2) is a heuristic choice, not derived from the data; a sensitivity check at other window widths was not performed and is noted as future work (Section 8). Recency weighting (older watches currently count equally with recent ones) and review-sentiment as a third signal were also not incorporated, left for a later iteration if the divergence signal needed refining — in the event, the signal as built proved sufficient to produce the clear ablation effect reported in Chapter 7, so this extension was not pursued.

Chapter 6

The Query Recommender Core

This chapter presents the recommender core in its current state of development. Consistent with the progress reported to the supervisor, the retrieval stage is currently being implemented; the remaining pipeline stages are planned as next steps and have not yet been built.

6.1 Overview

The recommender core is designed to combine three inputs — the current question, the user’s own past queries, and their behavioral profile (Chapter 5) — into three follow-up query suggestions. At the current stage, retrieval of semantically similar past queries is under implementation; candidate generation, schema-grounding validation, and ranking are planned as the next steps of this work and are not yet built.

6.2 Retrieval (In Progress)

Embedding-based retrieval is currently being implemented. Sentence embeddings and cosine similarity will be used to find previous user queries that are semantically related to the current question, even when they use different words. This retrieval step is intended to supply the similar-past-queries context used by the candidate-generation stage described below.

6.3 Planned Next Steps

The next step is to combine the current question, the retrieved similar past queries, and the behavioral profile (Chapter 5). An LLM will generate candidate follow-up questions from these three inputs; the candidates will then be validated against the domain schema, ranked, and diversified to select the best three. These stages are not yet implemented. Ollama is being used as a local LLM environment for development and testing.

Chapter 7

Evaluation

To be completed. The evaluation stage has not yet started, since the recommender core (Chapter 6) is still under development.

Chapter 8

Conclusion and Future Work

To be completed.

Bibliography

- [1] “Ontop — virtual knowledge graph system,” <https://ontop-vkg.org>.
- [2] I. Balasooriya, A. Dalla Vecchia, and E. Quintarelli, “Towards natural language reasoning and hybrid querying over virtual knowledge graphs,” in *ADBS 2026*, 2026.
- [3] A. Bacciu, E. Palumbo, A. Damianou, N. Tonellotto, and F. Silvestri, “Generating query recommendations via LLMs,” in *IR-RAG Workshop @ SIGIR 2024*, 2024.
- [4] D. Barman, R. Sarkar, and N. Chowdhury, “A cooperative co-evolutionary genetic algorithm for query recommendation,” *Multimedia Tools and Applications*, 2024.
- [5] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, 2024.